

Object Oriented Programming

Java



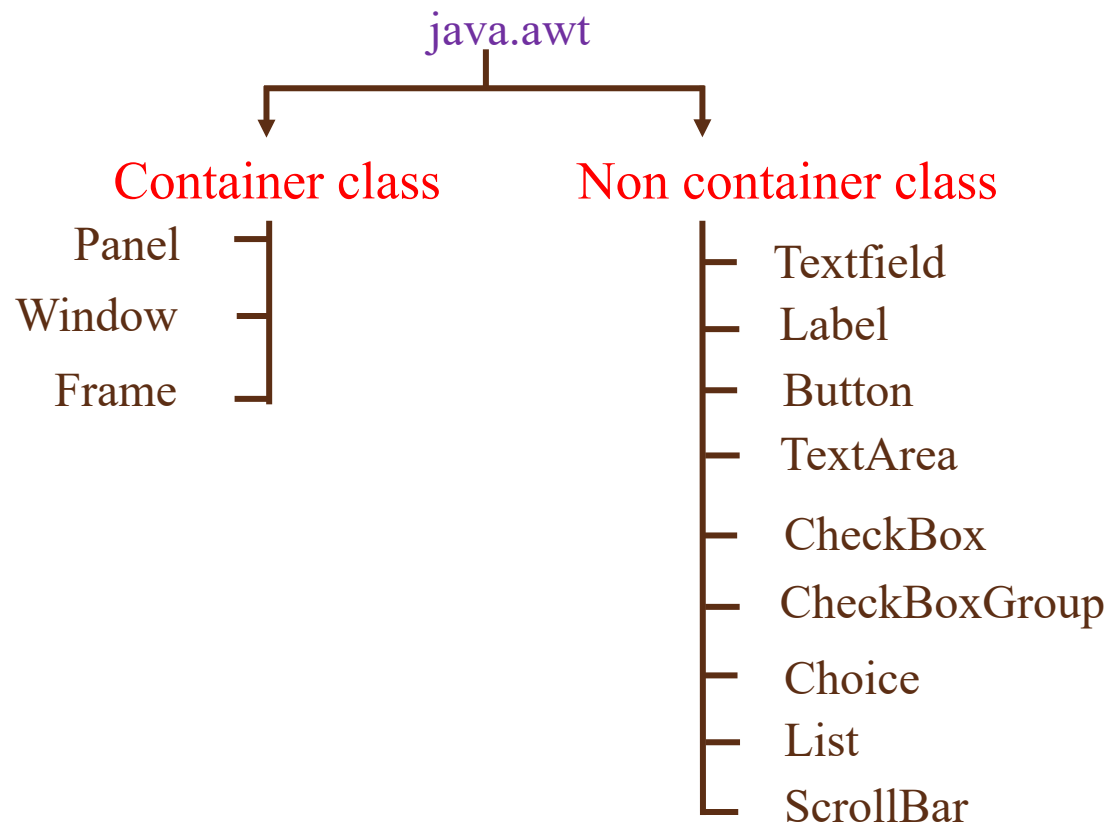
AWT in Java

- **Abstract Window Toolkit** (AWT) is an API to develop **GUI** or **window** – based applications in java.
- A graphical user interface is built of graphical elements called components. Typical components include such items as buttons, scrollbars, and text fields.
- Java AWT Components are **platform dependent** that is components are displayed according to the view of operating system.
- The **Container class is a subclass of Component**. Container class is used to display non-container class.
- In java, GUI can be designed using some predefined classes. All these classes are defined in **java.awt** package.

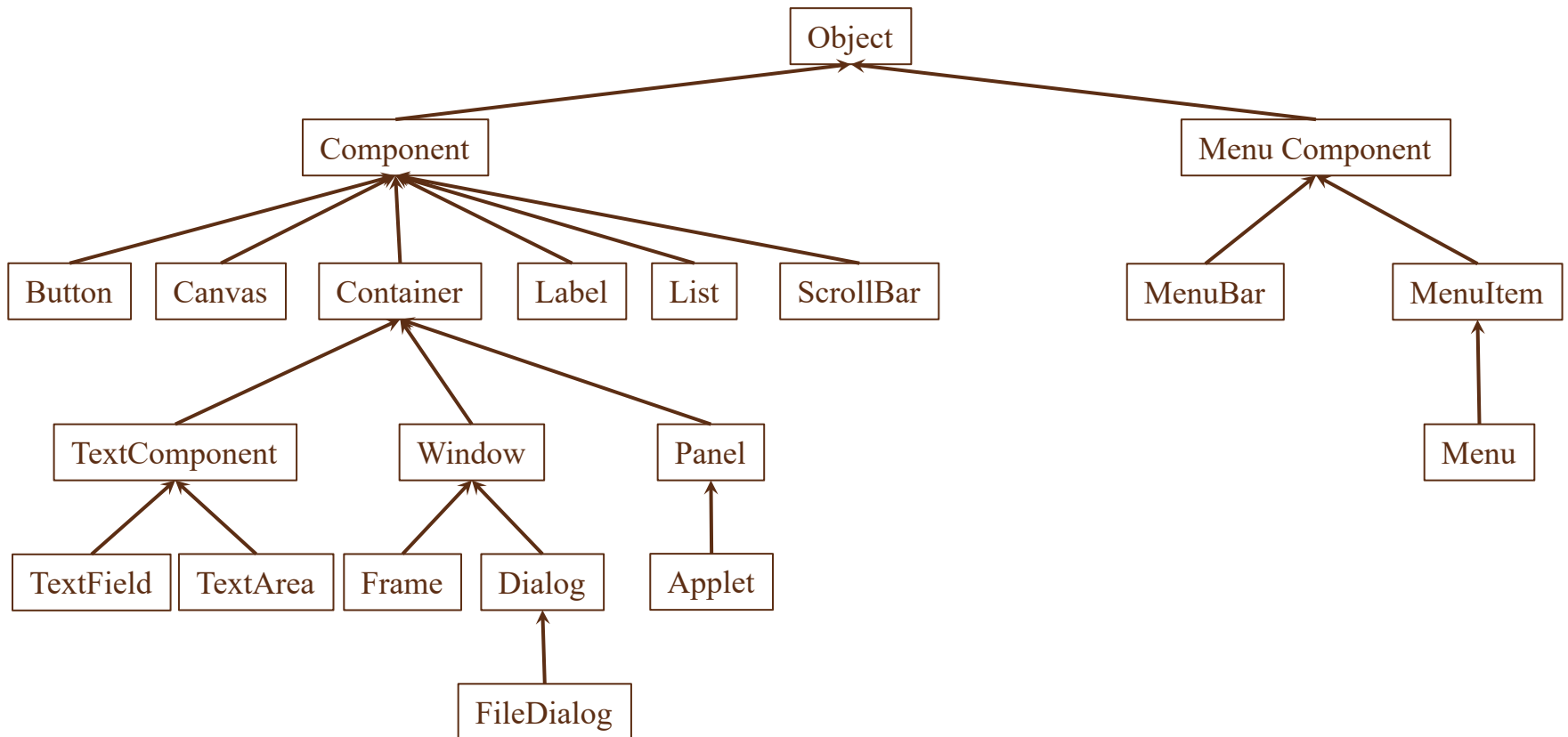


AWT in Java

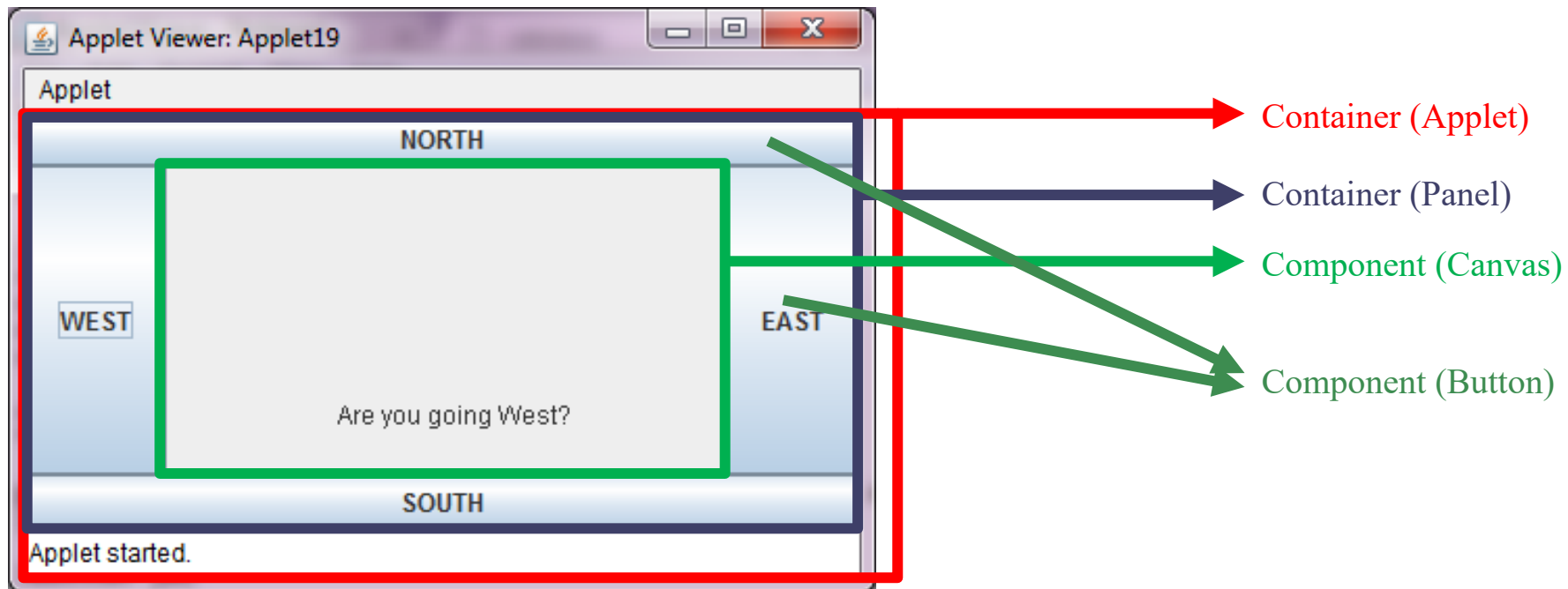
- **java.awt** class The AWT provides nine basic non-container component and three container classes.



Container Hierarchy



Example



Creating Applet

- Create a file named *SampleApplet.java* with the Java code shown here:

```
import java.applet.Applet;
```

```
import java.awt.Graphics;
```

```
public class HelloWorld extends Applet {
```

```
    public void paint(Graphics g) {
```

```
        g.drawString("CSE205-Java Technology", 50, 25);
```

```
    }
```

```
}
```

- Compile the Source File. It creates *SampleApplet.class* file.



Creating Applet

- Create an HTML File that Includes the Applet.

```
<html>
```

```
  <head>
```

```
    <title> A Simple Program </title>
```

```
  </head>
```

```
  <body>
```

```
    Here is the output of my program:
```

```
    <applet code="SampleApplet.class" width=150
```

```
height=25>
```

```
    </applet>
```

```
  </body>
```

```
</html>
```

- Run the Applet using a Java-compatible browser.



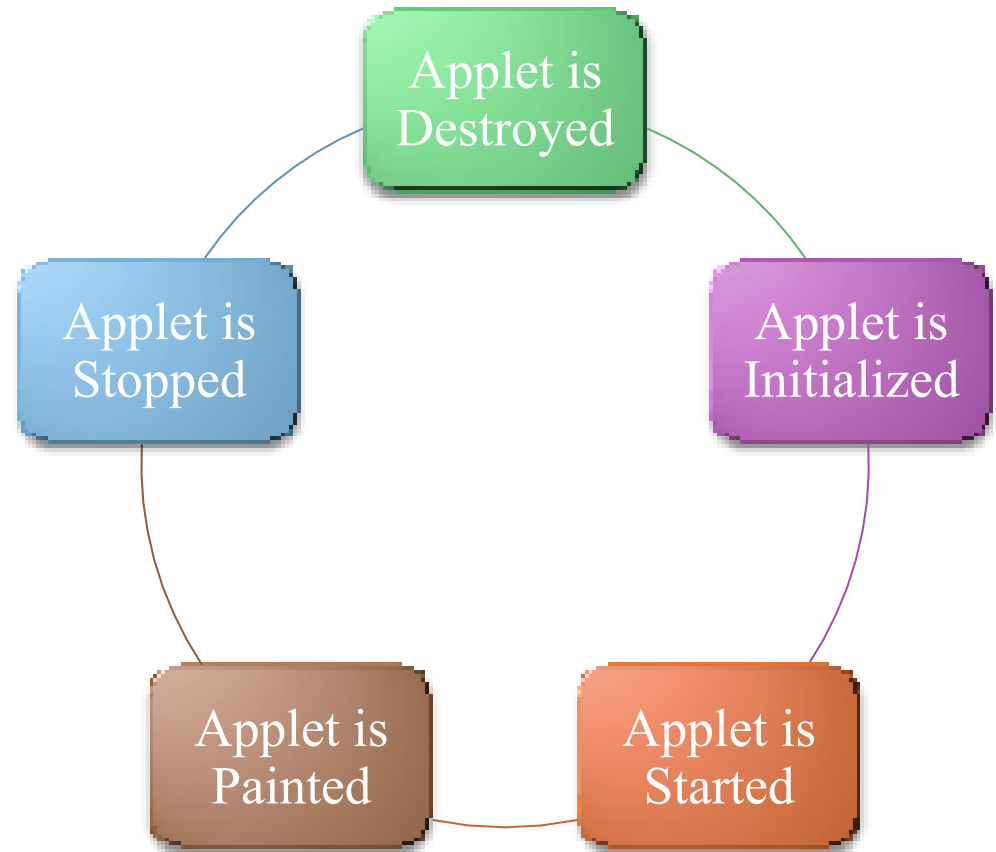
Basic Methods of Applet Class

- **void init():** This method is called once by the **browser** or **applet viewer**. It performs the initialization of an applet.
- **void start():** This method is called after the **init** method completes execution.
- **void paint(Graphics g):** This method is called automatically every time the applet needs to be **repainted**.
- **void stop():** This method is called by the **browser** or **applet viewer** when the applet should stop its execution.
- **void destroy():** This method is called by the **browser** or **applet viewer** when the applet is being removed from memory.



Lifecycle of Java Applet

1. Applet is initialized.
2. Applet is started.
3. Applet is painted.
4. Applet is stopped.
5. Applet is destroyed.



Creating Frame in AWT

- To create simple AWT example, you need a **frame**. There are two ways to create a frame in AWT.
 - By **extending** Frame class (inheritance)
 - By **creating the object** of Frame class (association)

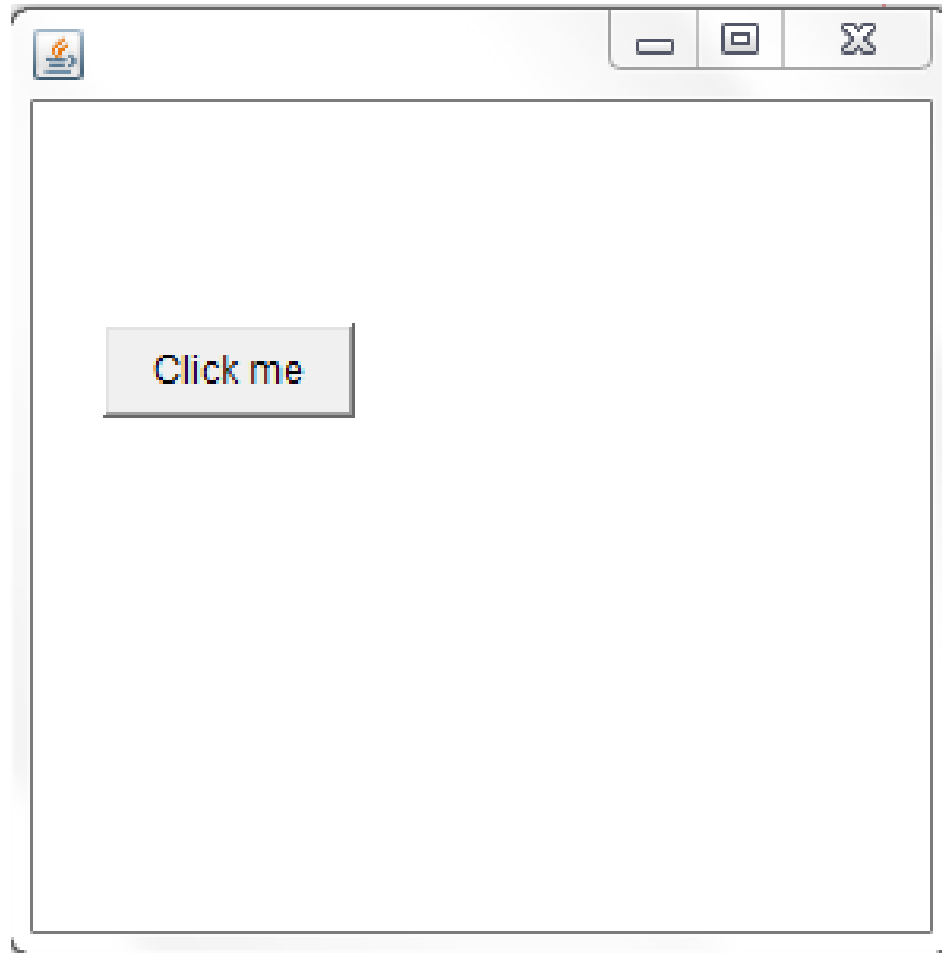


Extending Frame class

```
import java.awt.*;
public class First extends Frame{
    public First(){
        Button b = new Button("Click me");
        b.setSize(80, 30);
        b.setLocation(30, 100); // setting button position
        add(b);                //adding button into frame
        setSize(300,300);      //frame size 300 width and 300 height
        setLayout(null);       //no layout manager
        setVisible(true);      //now frame will be visible, by default not visible
    }
    public static void main(String args[]){
        First f = new First();
    }
}
```



Extending Frame class



Creating Object of Frame Class

```
import java.awt.*;
public class First2 {
    public First2() {
        Frame f=new Frame();
        Button b=new Button("Click me");
        b.setSize(80, 30);
        b.setLocation(30, 100);
        f.add(b);
        f.setSize(300,300);
        f.setLayout(null);
        f.setVisible(true);
    }
    public static void main(String args[]) {
        First2 f=new First2();
    }
}
```



Frame Methods

Method	Description
<code>public void add(Component c)</code>	inserts a component on this component.
<code>public void setSize(int width,int height)</code>	sets the size (width and height) of the component.
<code>public void setLayout(LayoutManager m)</code>	defines the layout manager for the component.
<code>public void setVisible(boolean status)</code>	changes the visibility of the component, by default false.
<code>public setBackground(color)</code>	sets the background color.
<code>public setForeground(color)</code>	sets the foreground or text color.



Event Handling

Event: An event is an Object that describes a state change in a source.

Some of the activities that causes event to be generated are:

- Pressing a button
- Pressing a key on Keyboard
- Selecting an item from a list.

Event Source: A source is an object that generates an event. Some general event sources are,

- Buttons
- Checkbox
- List
- Text Items



Event Handling

Event Listener: A listener is an object that is notified when an event occurs.

For example:

mouseMotionListener *interface* defines two events;
when mouse is moved
when mouse is dragged



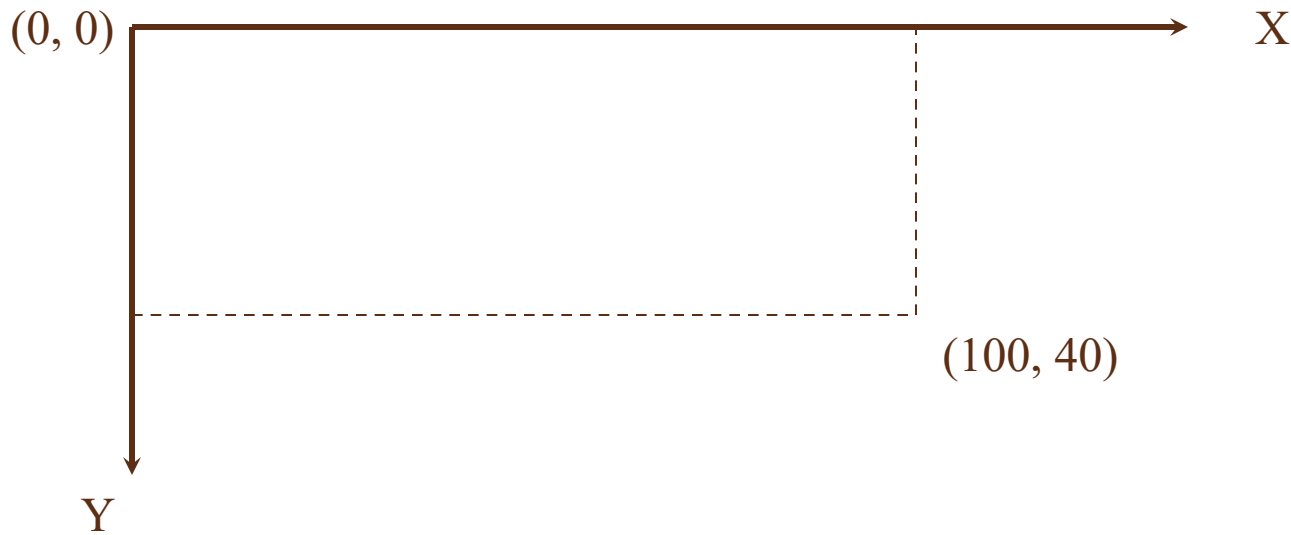
Graphics Programming

Drawing Shapes

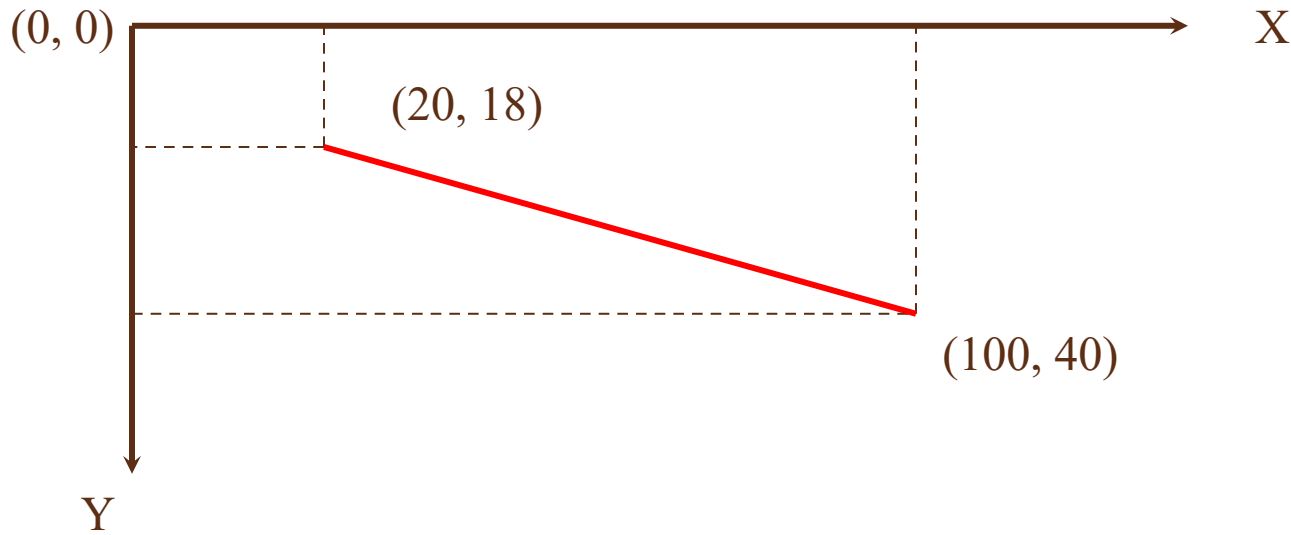


Coordinate Systems

- Each pixel can be identified using a two-dimensional coordinate system.
- When referring to a pixel in a Java program, we use a coordinate system with the origin in the top-left corner.

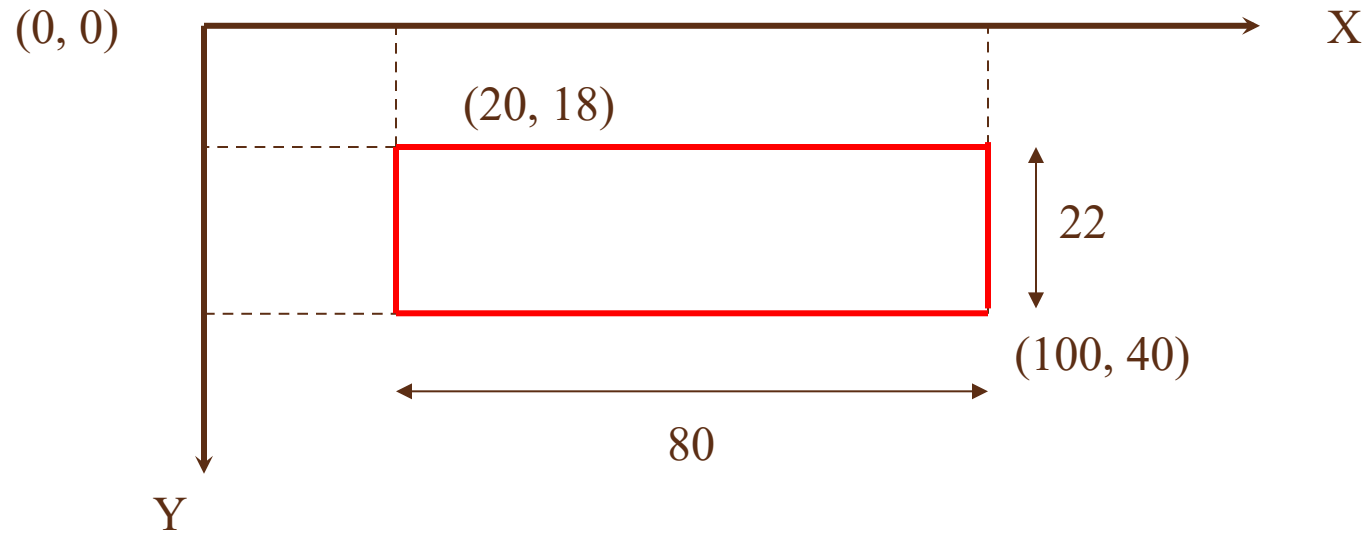


Drawing a Line



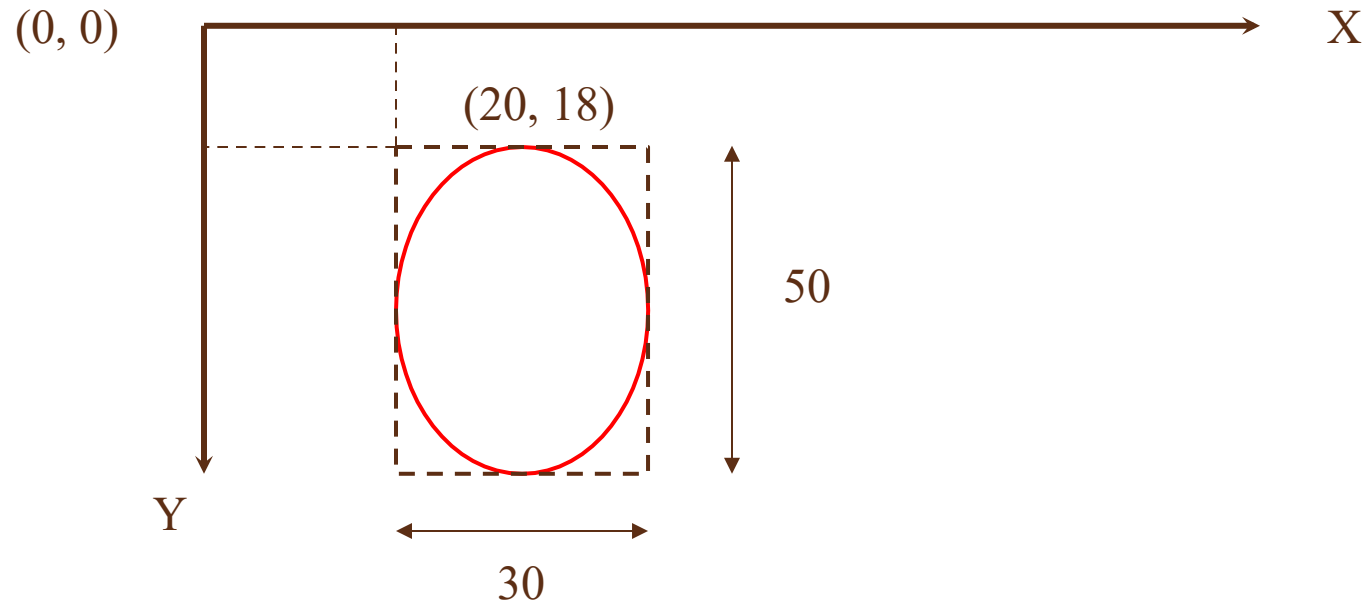
```
g.drawLine(int startX, int startY, int endX, int endY);  
g.drawLine(20, 18, 100, 40);
```

Drawing a Rectangle



```
g.drawRect(int top, int left, int width, int height);  
g.drawRect(20, 18, 80, 22);
```

Drawing an Oval



```
g.drawOval(int top, int left, int width, int height);  
g.drawOval(20, 18, 30, 50);
```

Add Components

Let a frame be created; `Frame f = new Frame();`

- **Label:** `Label lb = new label(“User Name”, Label.Left);`
`f.add(lb);`
- **Button:** `Button b = new Button(“OK”);`
`f.add(b);`
- **List:** `List lt = new List();`
`lt.add(“Category1”);`
`lt.add(“Category2”);`
`f.add(lt);`
- **TextField:** `TextField txf = new TextField();`
`f.add(txf);`



Layout Managers

- Since the component class defines the setSize() and setLocation() methods, all components can be sized and positioned with those methods. It defines in term of pixels.
- It creates problem when the platform changes.
- Solution: Layout Manager.
- Layout Manager determines the size and placement of the components.



Layout Managers

- Several different Layout Managers each bases on an algorithm:
 - FlowLayout
 - BorderLayout
 - GridLayout
- For window, frame default layout is BorderLayout and for panel default layout is FlowLayout.



FlowLayout

- The algorithm used by the FlowLayout is to lay out Components like words on a page: Left to right, top to bottom.
- It fits as many Components into a given row before moving to the next row.

```
Panel aPanel = new Panel();  
aPanel.add(new Button("Ok"));  
aPanel.add(new Button("Add"));
```



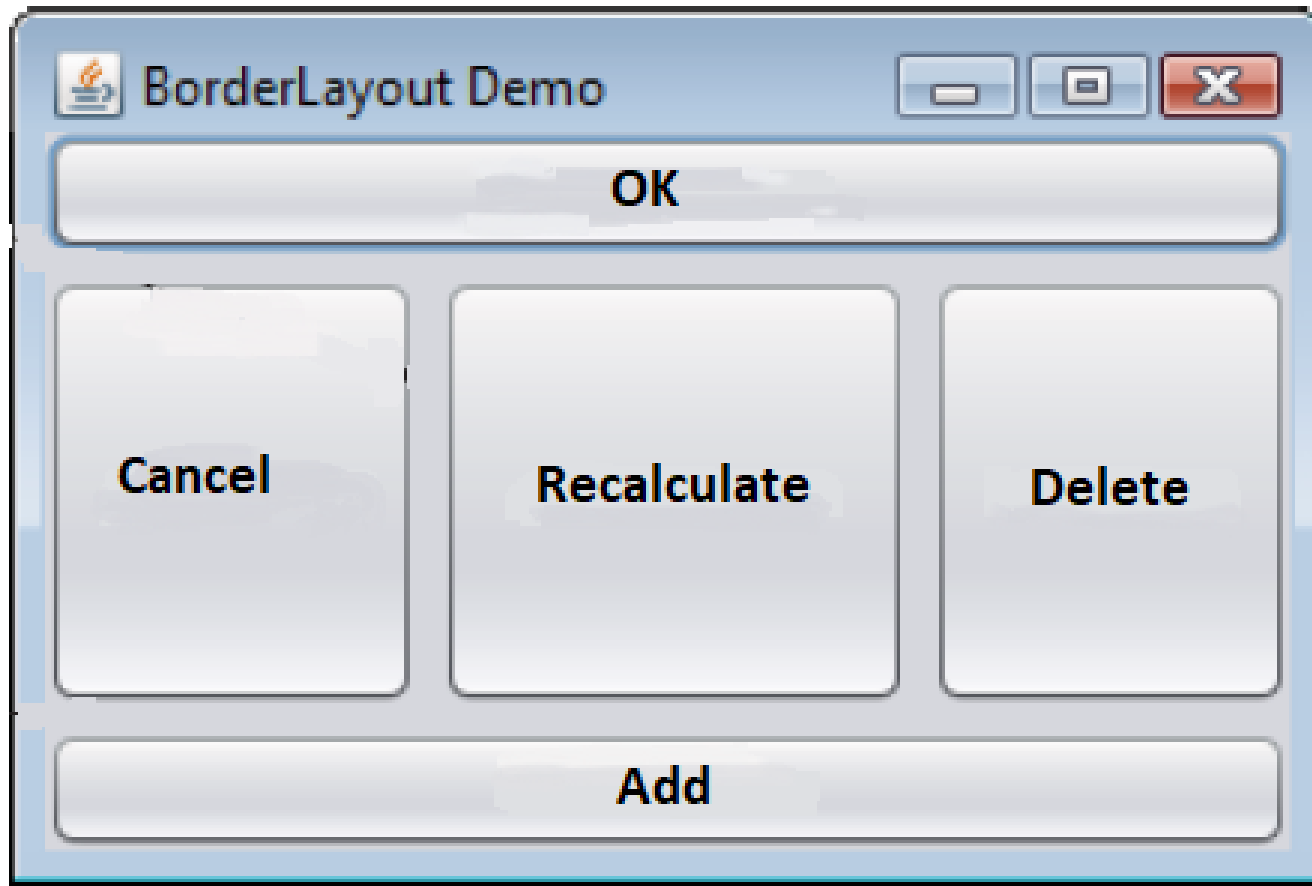
BorderLayout

- The BorderLayout Manager breaks the Container up into 5 regions (North, South, East, West, and Center).
- When Components are added, their region is also specified.

```
Frame aFrame = new Frame();  
aFrame.add("North", new Button("Ok"));  
aFrame.add("South", new Button("Add"));  
aFrame.add("East", new Button("Delete"));  
aFrame.add("West", new Button("Cancel"));  
aFrame.add("Center", new Button("Recalculate"));
```



BorderLayout



GridLayout

- The GridLayout class divides the region into a grid of equally sized rows and columns.
- Components are added left-to-right, top-to-bottom.
- The number of rows and columns is specified in the constructor for the LayoutManager.

```
Panel aPanel = new Panel();  
GridLayout theLayout = new GridLayout(2,2);  
aPanel.setLayout(theLayout);  
aPanel.add(new Button("Ok"));  
aPanel.add(new Button("Add"));  
aPanel.add(new Button("Delete"));  
aPanel.add(new Button("Cancel"));
```



GridLayout

